



You built your first Spring Boot app. Now learn how production applications actually work.

Spring Boot — Next Level

Security → Performance → Deployment → Scale

Build Real Applications After Basics



SECURITY



PERFORMANCE



DEPLOYMENT



SCALE

Request Flow **Inside Spring Boot**

 Every API request follows the same journey.



Client

Initiates request



Controller

Accepts request



Service

Applies business rules



Repository

Queries data



Database

Stores/retrieves data



Response

Returns JSON to client

```
</> GET /users → Controller → Service → DB → JSON
```

Master this flow before learning advanced topics.

DTO vs Entity



Real applications separate API models from database models.

ENTITY

- Maps database tables
- Contains persistence logic

DATABASE LAYER

DTO

- Controls request/response structure
- Hides internal fields

API/CLIENT LAYER

UserEntity ↓ UserDTO

Never expose entities directly.

REST API Design



Good APIs feel predictable.

GET

→ Fetch

POST

→ Create

PUT

→ Update

DELETE

→ Remove

BEST PRACTICES

✓ Use nouns

✓ Use status codes

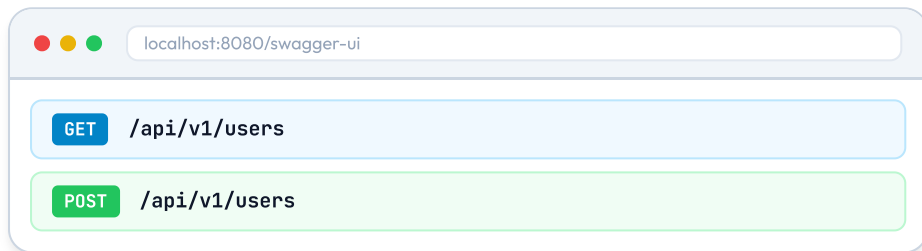
✓ Version APIs

 </api/v1/users>

Design for future changes.

API Documentation

? Good documentation reduces developer questions.



Use Swagger & OpenAPI

Auto-generated docs | Try APIs in browser | Share endpoints instantly

Document while building.

AuthN vs AuthZ



These solve two different problems.



Authentication (Who are you?)

Examples: Login | JWT | OAuth



Authorization (What can you do?)

Examples: USER | ADMIN | Permissions

→ Identity → Access

Login does not mean access.



Spring Security Basics



Security becomes difficult if added later.

1 Security Filter Chain

2 JWT Authentication

3 Password Encoding

4 Role Permissions

Request → Validate Token → Grant Access

Secure endpoints early.

Pagination & Sorting



Performance starts with loading less data.

USE: PAGEABLE | SORT | PAGE

Page 1 (Records 1-10)

Active Page

Page 2 (Records 11-20)

Database Page



BENEFITS (EXAMPLE: ?PAGE=1&SIZE=10)



Lower memory



Faster APIs



Better UX

Never return huge lists.

Database Caching

📖 Databases are expensive. Cache reduces unnecessary queries.

Client



Application

Hit (Fast)

Cache (Redis)

Miss
→

Database



LEARN

Spring Cache

Redis

@Cacheable

@CacheEvict



USE CASES

- ✓ User profiles
- ✓ Product catalog
- ✓ Dashboard & APIs



BENEFITS



Lower DB load



Faster response



Scalability

Cache data that changes less often and gets requested frequently.

Scheduling Jobs



Some work happens automatically.

@Scheduled

Generate reports

@Scheduled

Send emails

@Scheduled

Cleanup logs

Cron Example: 0 0 9 * * *

Automate repetitive tasks.

Async Processing

⚙️ Users should not wait for slow operations.

MAIN THREAD

NON-BLOCKING RESPONSE

ASYNC THREAD

EMAIL | PDF | UPLOADS



@Async & CompletableFuture

Benefit: Provides a completely non-blocking user experience.

Move heavy work to background.

Event-Driven Architecture



Modern applications communicate using events.

Order Created

✉ Event Bus / Broker

Payment Service

Notification Service

BENEFITS



Loose coupling



Scalability



Independent services

Events reduce dependencies.



DATA STORAGE

File Upload & Storage



Uploading files is easy. Managing them properly is not.



Upload



Validate



Store File



Metadata



STORAGE OPTIONS



Local → Dev



Cloud → Prod



DB → Small files



GOOD PRACTICES



Restrict size



Valid extensions



Unique names



LEARN

MultipartFile

File validation

Streaming

Storage strategies

Store files outside databases whenever possible.



Exception Standards

⚠ Errors should explain what failed and what to do next.

❌ BAD RESPONSE

"Something went wrong"

✅ GOOD RESPONSE

```
{  
  "status": 404,  
  "message": "User not found",  
  "code": "USER_001"  
}
```

🔧 SPRING TOOLS

@ControllerAdvice
@ExceptionHandler
ResponseBody

✅ BENEFITS

- ✓ Cleaner API schemas
- ✓ Easier app debugging
- ✓ Consistent client logic

Users should never see raw exceptions.



Profiles & Secrets



Production configuration should never live inside source code.

`</>` Code



Profiles



Secure Secrets



PROFILES

```
application-dev.yml
application-test.yml
application-prod.yml
```

Activate: `spring.profiles.active`



SECRETS STORAGE

- ✓ Env Variables
- ✓ Secret Managers
- ✓ Container configs

`</>` Code changes rarely.

⚙️ Config changes often.

Separate code, configuration, and secrets.



Docker & Spring Boot



Containers remove deployment surprises.

1. **CODE** Spring application code
2. **DOCKERFILE** Container config script
3. **IMAGE** Packaged template image
4. **CONTAINER** Running isolated container

Benefits: Consistency | Portability | Easy deployment

Package once, run anywhere.

CI/CD & Deployment

∞ Writing code is only half the job. Deployment delivers value.

</>
Commit

↗
Build

🧪
Test

📦
Container

🚀
Deploy



DEPLOYMENT OPTIONS

Jenkins

Traditional & older enterprise CI/CD.

GitHub Actions

Modern pipeline from repository.

Argo Workflows

Orchestrates complex build pipelines.

Argo CD

GitOps Kubernetes sync.



CLOUD DEPLOYMENT



GCP → Cloud Run / GKE



Azure → App Service / AKS



PROD FLOW

Spring Boot

✓ Docker

✓ Argo CD

✓ Kubernetes

✓ GCP / Azure

Modern teams automate deployment and treat infrastructure like code.

Monitoring Production

👁️ Healthy systems are observable.

LEARN: ACTUATOR | LOGS | METRICS | TRACING

IS APP ALIVE?

Actuator Alive

PERFORMANCE / ERRORS

No Failures

QUESTIONS TO ANSWER



Is app alive?



Why slow?



What failed?

Monitoring reduces downtime.

Build Your **First Project**

 Projects create understanding.



Expense Tracker

Manage transactions and budgets



Task Manager

Assign and prioritize tasks



Inventory API

Track items and stock counts



Blog Backend

REST API with role commenting

Must include: Security | Database | Docker | Testing

Ship projects.

 NEXT STEPS

What to Learn **After This**



You are moving beyond CRUD applications.



Microservices Architecture



Kafka Message Streaming



Kubernetes Orchestration



Cloud Platforms & System Design

Learn → Build → Deploy → Scale